LAB RECORD

PYTHON PROGRAMS

Course Code : CSA17
Course Name : ARTIFICIAL INTELLIGENCE

Name : R.Kishan
Reg No : 192525378
SIMATS Engineering

INDEX

S.No	Name of the Program
1	8-Puzzle Problem
2	8-Queens Problem
3	Water Jug Problem
4	Crypt-Arithmetic Problem
5	Missionaries and Cannibals Problem
6	Vacuum Cleaner Problem
7	Breadth First Search (BFS)
8	Depth First Search (DFS)
9	Travelling Salesman Problem
10	A* Algorithm
11	Map Coloring (Constraint Satisfaction Problem)
12	Tic Tac Toe Game
13	Minimax Algorithm
14	Alpha-Beta Pruning
15	Decision Tree (ID3)
16	Feed-Forward Neural Network

Program 1: 8-Puzzle Problem

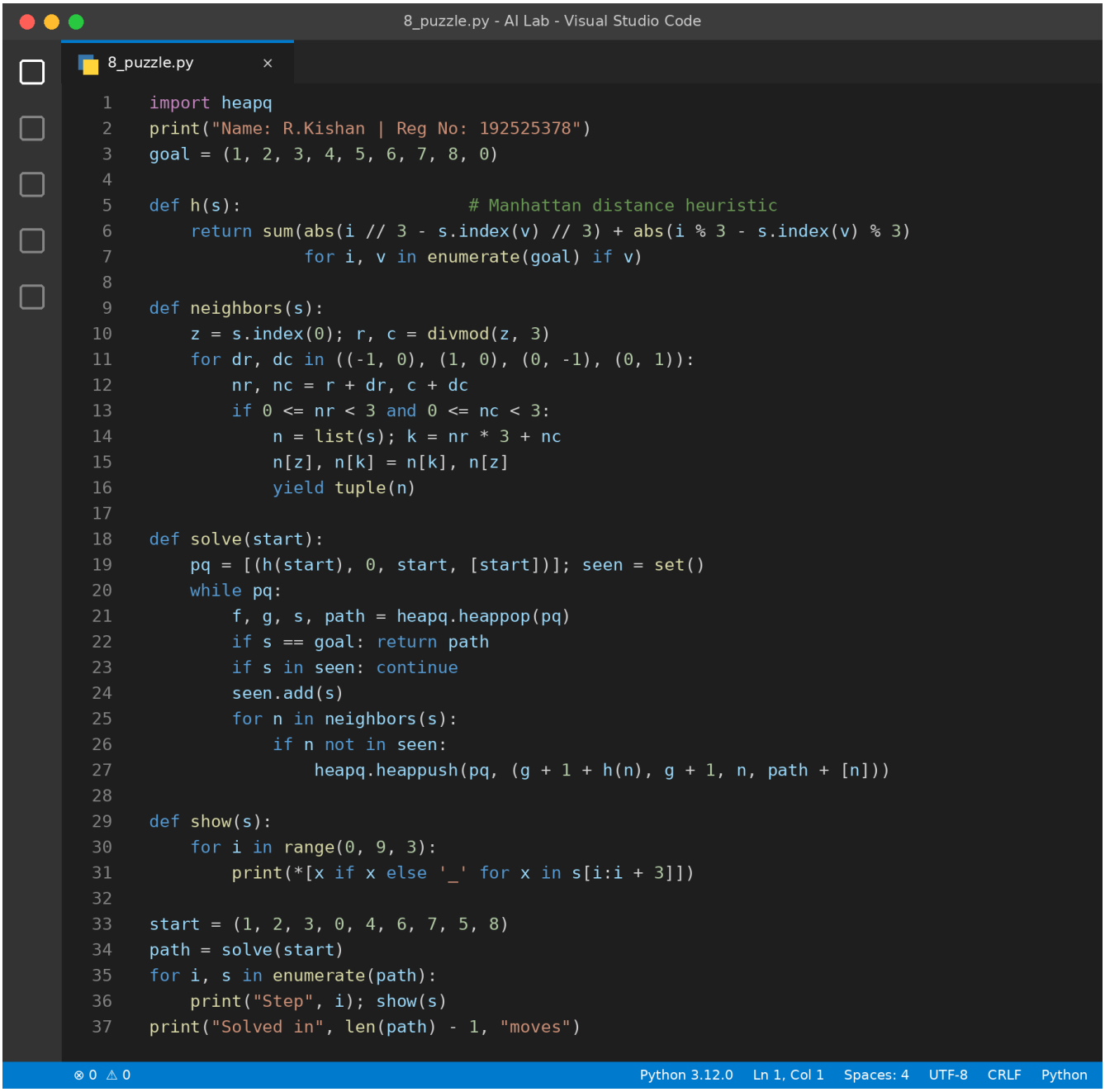
Aim

To write a Python program to solve the 8-Puzzle problem using the A* search algorithm with the Manhattan-distance heuristic.

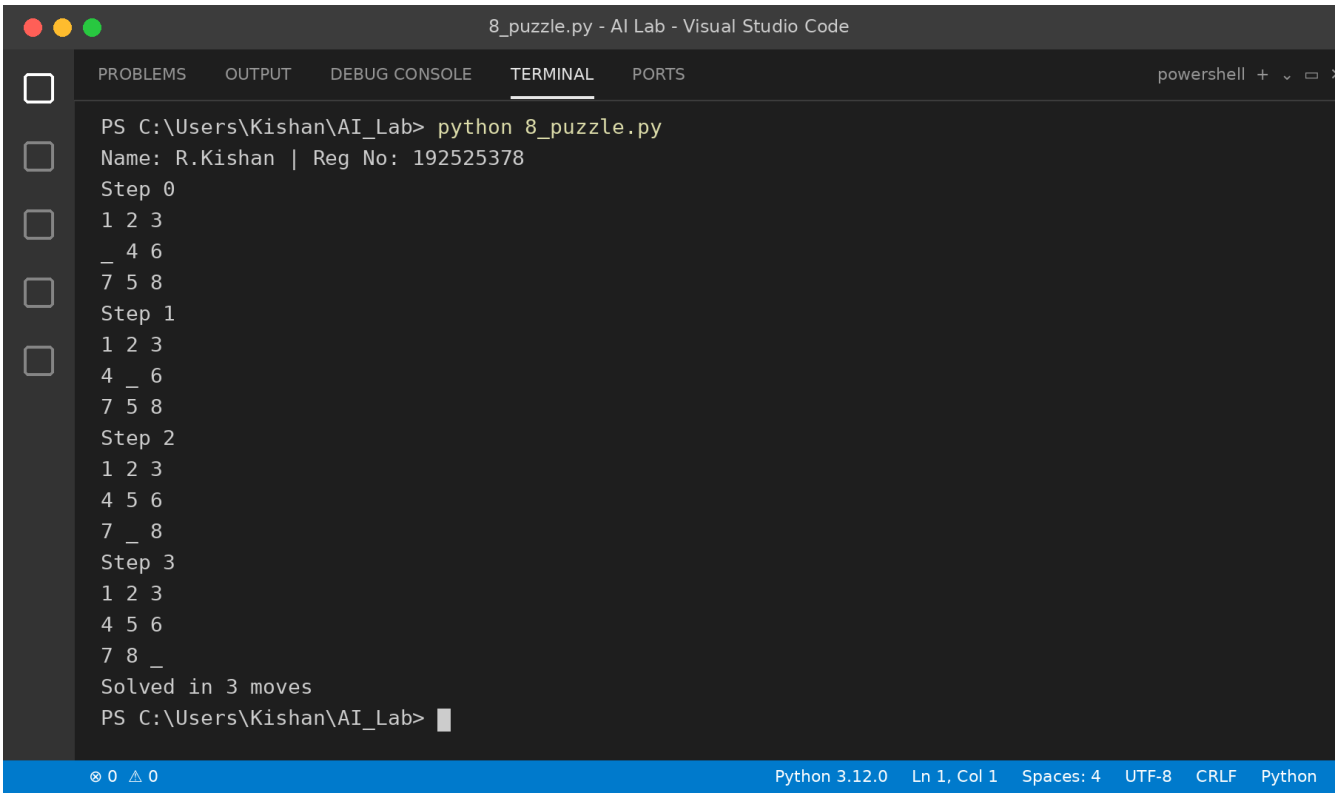
Procedure

1. Represent each board state as a tuple of 9 numbers, where 0 is the blank tile.
2. Define the goal state (1 2 3 / 4 5 6 / 7 8 _) and the heuristic h(n) as the sum of Manhattan distances of all tiles.
3. Generate the neighbours of a state by sliding the blank up, down, left and right.
4. Keep a priority queue ordered by f(n) = g(n) + h(n) and always expand the state with the lowest f value.
5. Stop when the goal state is removed from the queue and print every board along the solution path.

Program



Output



Result: Thus the Python program for 8-Puzzle Problem was executed successfully and the output was verified.

Program 2: 8-Queens Problem

Aim

To write a Python program to place 8 queens on an 8x8 chessboard so that no two queens attack each other, using backtracking.

Procedure

1. Place one queen per row and store the column of each queen in a list.
2. For a new queen, check that its column and both diagonals are free (safe function).
3. Try every column of the current row; if a column is safe, place the queen and move to the next row.
4. If no column is safe, backtrack: remove the previous queen and try the next column.
5. When all 8 rows are filled, store the solution; finally print the number of solutions and the first board.

Program

8_queens.py - AI Lab - Visual Studio Code

8_queens.py

```python
print("Name: R.Kishan | Reg No: 192525378")
N = 8
solutions = []

def safe(cols, r, c):
    return all(c != cc and abs(c - cc) != r - rr for rr, cc in enumerate(cols))

def place(cols):
    r = len(cols)
    if r == N:
        solutions.append(cols[:]); return
    for c in range(N):
        if safe(cols, r, c):
            cols.append(c); place(cols); cols.pop()

place([])
print("Total solutions:", len(solutions))
print("First solution :", solutions[0])
for c in solutions[0]:
    print(' '.join('Q' if i == c else '.' for i in range(N)))
```

⊗ 0 ⚠ 0 Python 3.12.0 Ln 1, Col 1 Spaces: 4 UTF-8 CRLF Python

Output

8_queens.py - AI Lab - Visual Studio Code

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell

```
PS C:\Users\Kishan\AI_Lab> python 8_queens.py
Name: R.Kishan | Reg No: 192525378
Total solutions: 92
First solution : [0, 4, 7, 5, 2, 6, 1, 3]
Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . Q . . . .
PS C:\Users\Kishan\AI_Lab>
```

⊗ 0 ⚠ 0 Python 3.12.0 Ln 1, Col 1 Spaces: 4 UTF-8 CRLF Python

Result: Thus the Python program for 8-Queens Problem was executed successfully and the output was verified.

Program 3: Water Jug Problem

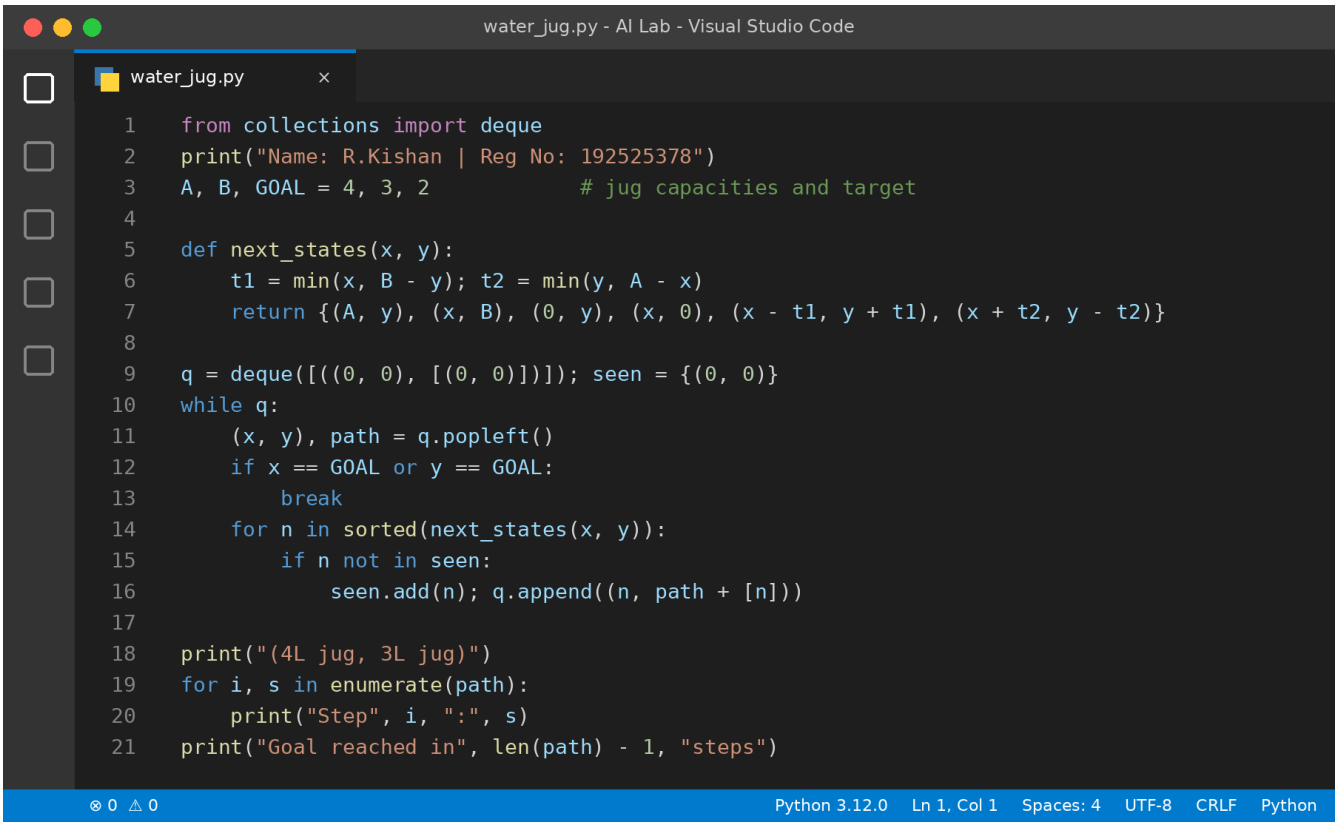
Aim

To write a Python program to measure exactly 2 litres of water using a 4-litre jug and a 3-litre jug, using Breadth First Search.

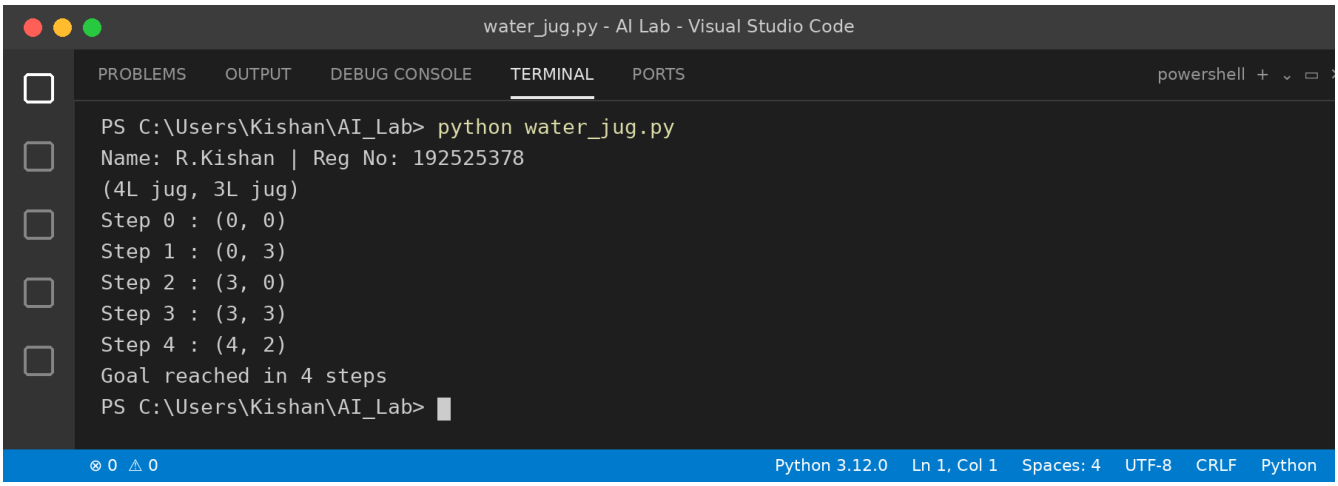
Procedure

1. Represent a state as (x, y) where x is water in the 4L jug and y is water in the 3L jug; start at (0, 0).
2. Define the operations: fill a jug, empty a jug, and pour from one jug into the other until full or empty.
3. Use a queue to explore states level by level and a visited set to avoid repeating states.
4. Stop when either jug contains the target quantity (2 L).
5. Print the sequence of states from the start to the goal.

Program



Output



Result: Thus the Python program for Water Jug Problem was executed successfully and the output was verified.

Program 4: Crypt-Arithmetic Problem

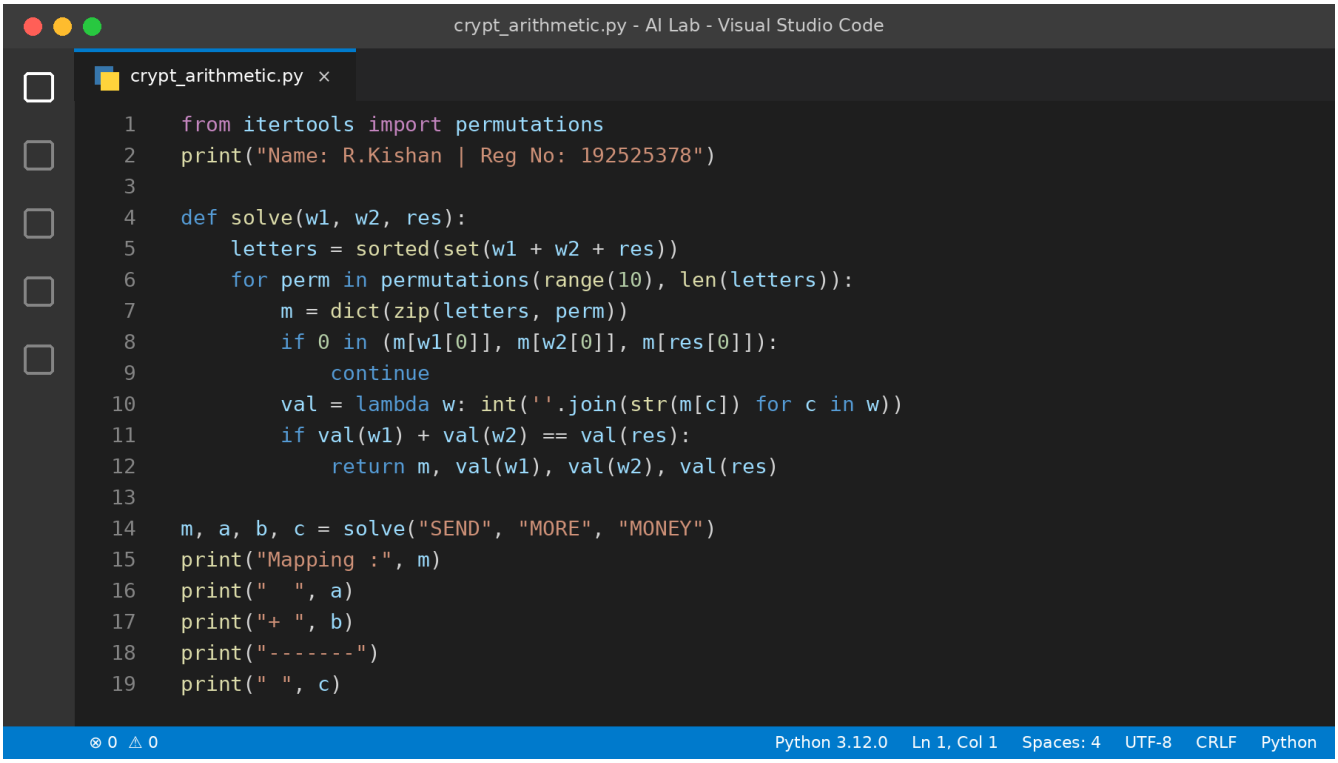
Aim

To write a Python program to solve the crypt-arithmetic puzzle SEND + MORE = MONEY by assigning a unique digit to every letter.

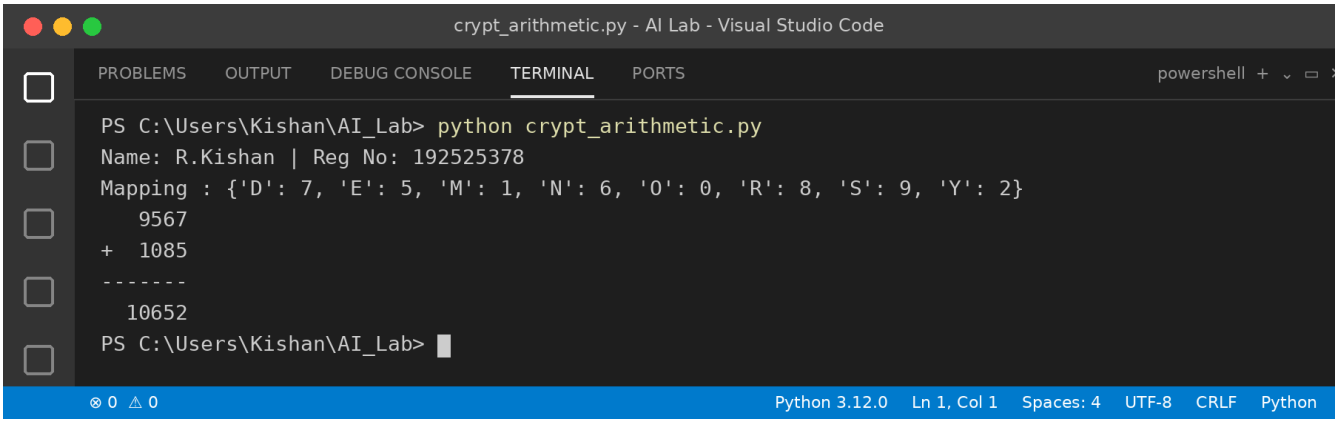
Procedure

1. Collect all distinct letters of the three words.
2. Generate every permutation of digits 0-9 of the same length as the number of letters.
3. Reject any assignment in which the leading letter of a word is zero.
4. Convert the words to numbers and test whether SEND + MORE = MONEY.
5. Print the first assignment that satisfies the equation.

Program



Output



Result: Thus the Python program for Crypt-Arithmetic Problem was executed successfully and the output was verified.

Program 5: Missionaries and Cannibals Problem

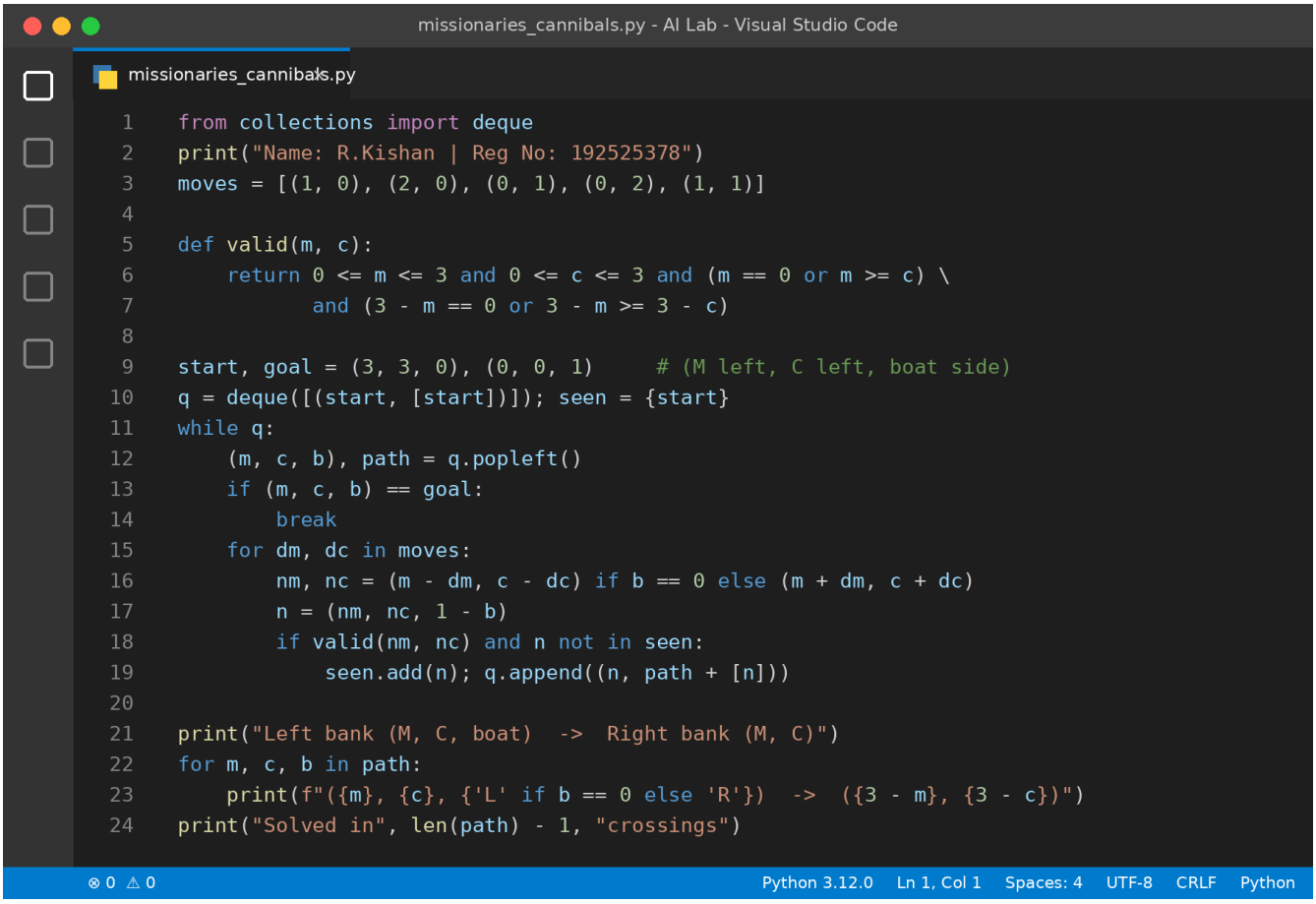
Aim

To write a Python program to move 3 missionaries and 3 cannibals across a river with a 2-person boat so that cannibals never outnumber missionaries on either bank.

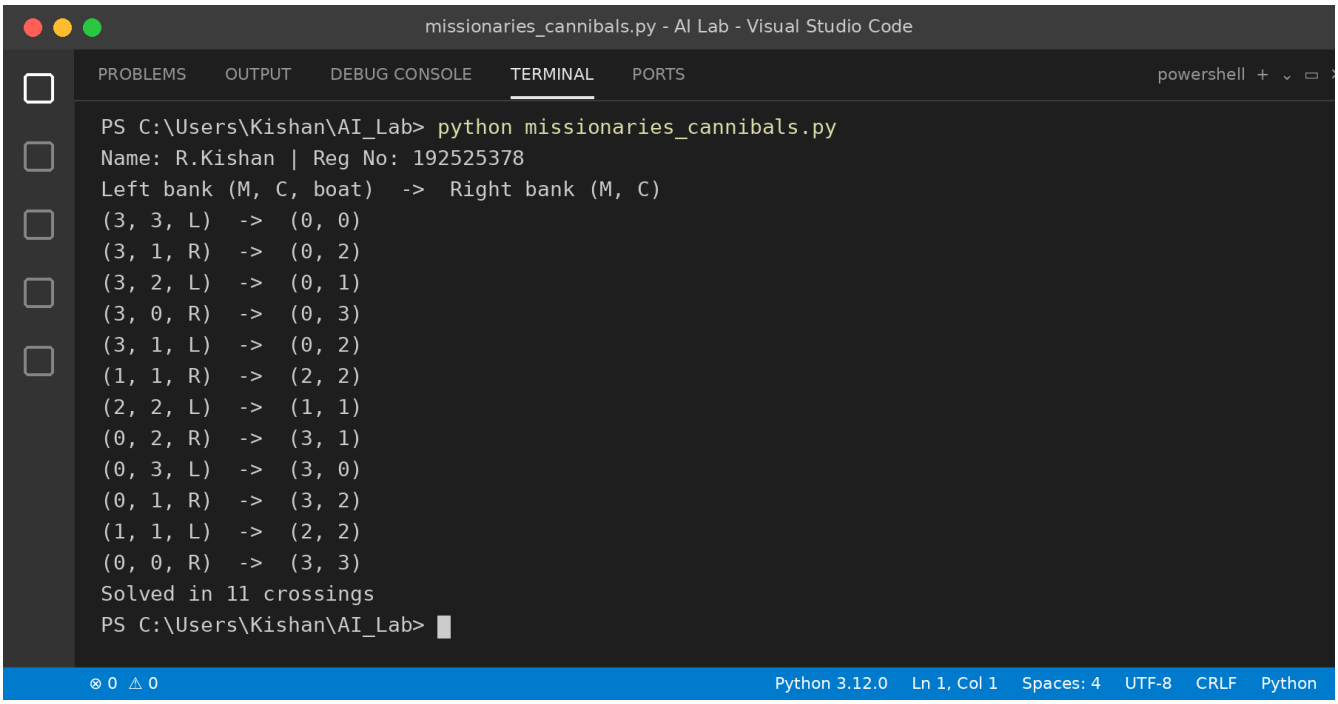
Procedure

1. Represent a state as (missionaries on left, cannibals on left, boat position).
2. The boat can carry (1,0), (2,0), (0,1), (0,2) or (1,1) people per trip.
3. A state is valid only if missionaries are not outnumbered on either bank (unless there are none on that bank).
4. Use Breadth First Search from (3,3,Left) until the goal (0,0,Right) is reached.
5. Print each state of the shortest solution.

Program



Output



Result: Thus the Python program for Missionaries and Cannibals Problem was executed successfully and the output was verified.

Program 6: Vacuum Cleaner Problem

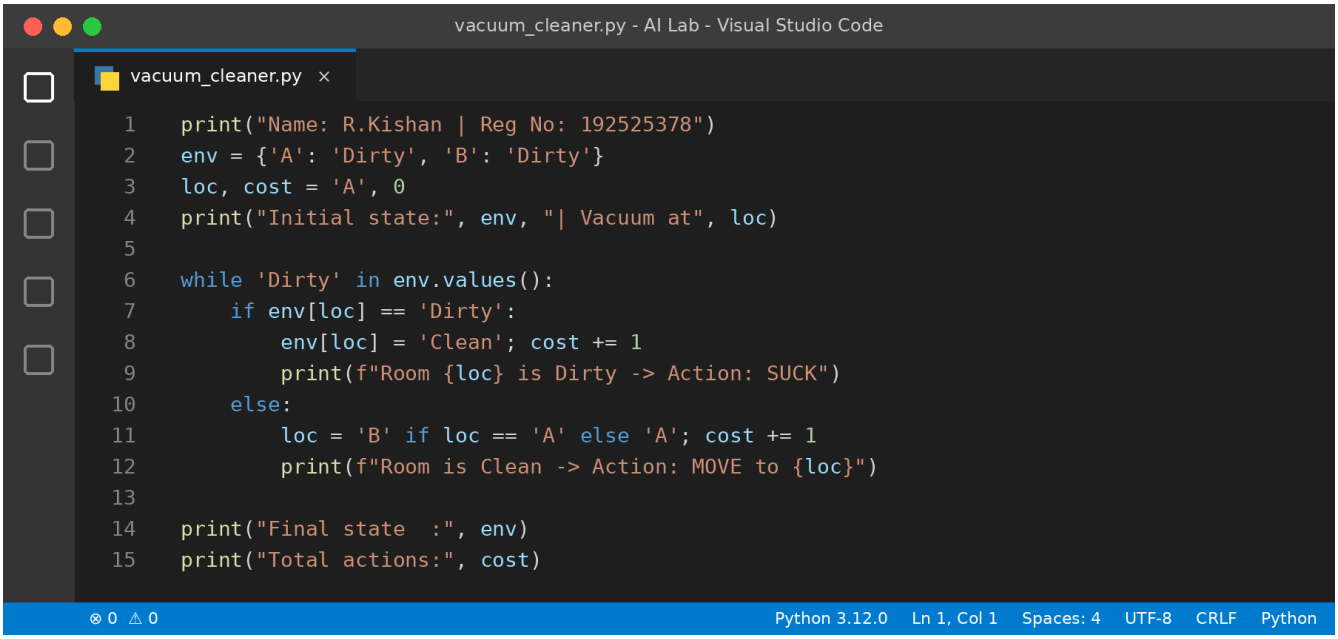
Aim

To write a Python program that simulates a simple reflex vacuum-cleaner agent in a two-room world (rooms A and B).

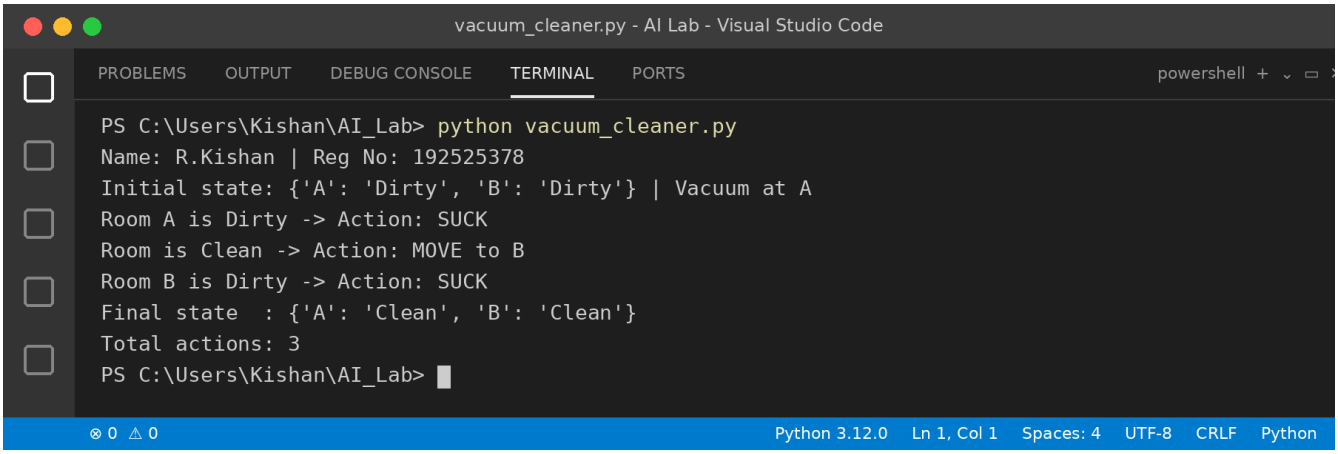
Procedure

1. Create the environment with the status (Clean/Dirty) of rooms A and B and place the agent in room A.
2. Percept: the current location and the status of that location.
3. Rule: if the current room is Dirty then SUCK; otherwise MOVE to the other room.
4. Repeat until both rooms are clean, counting every action performed.
5. Print the action taken at each step, the final state and the total number of actions.

Program



Output



Result: Thus the Python program for Vacuum Cleaner Problem was executed successfully and the output was verified.

Program 7: Breadth First Search (BFS)

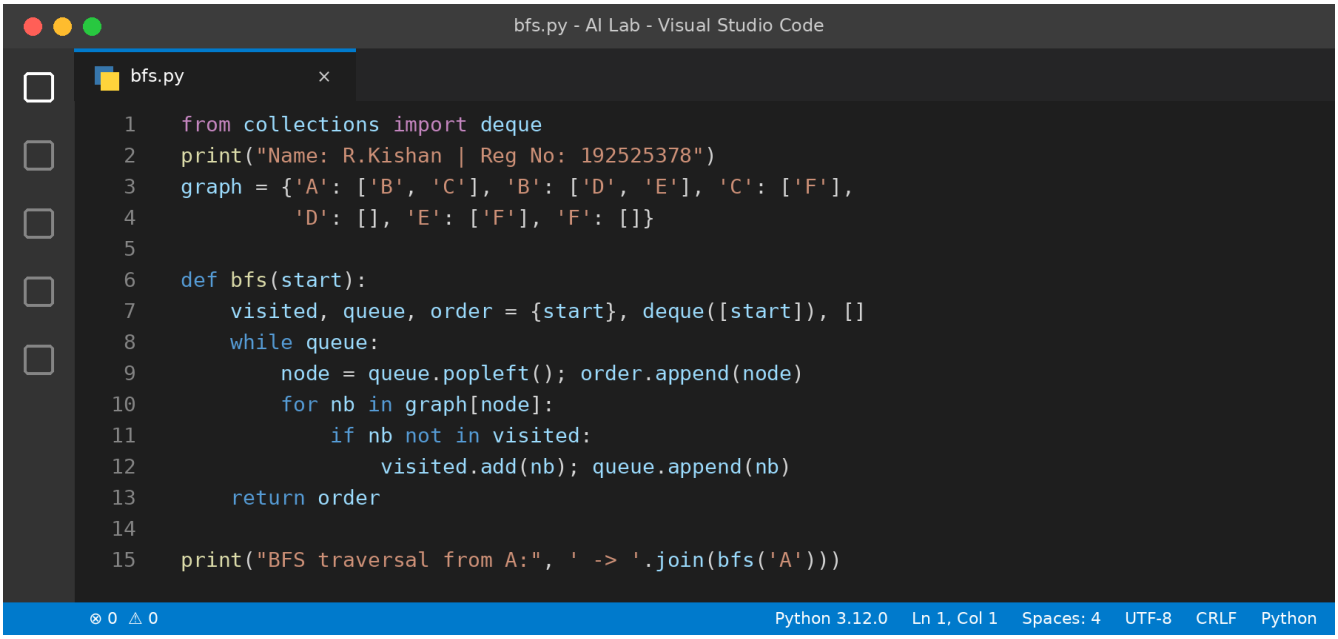
Aim

To write a Python program to traverse a graph using Breadth First Search.

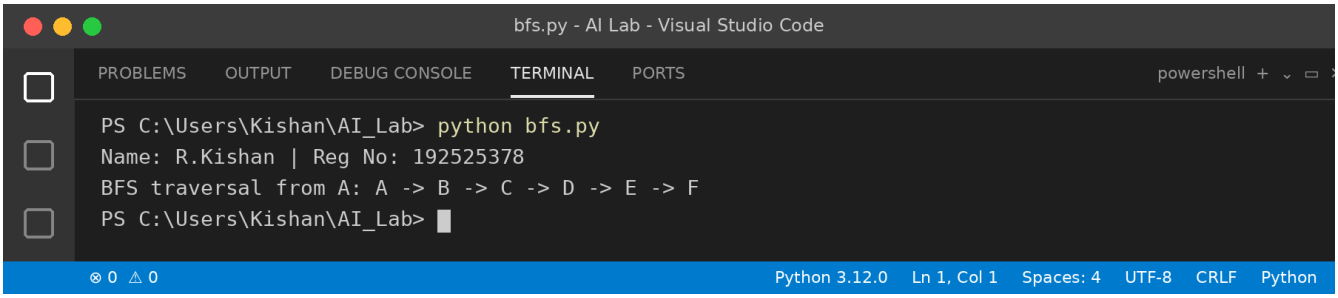
Procedure

1. Represent the graph as an adjacency list (dictionary).
2. Insert the start node into a queue and mark it as visited.
3. Remove the front node from the queue, add it to the traversal order and insert all its unvisited neighbours.
4. Repeat until the queue becomes empty.
5. Print the order in which the nodes were visited.

Program



Output



Result: Thus the Python program for Breadth First Search (BFS) was executed successfully and the output was verified.

Program 8: Depth First Search (DFS)

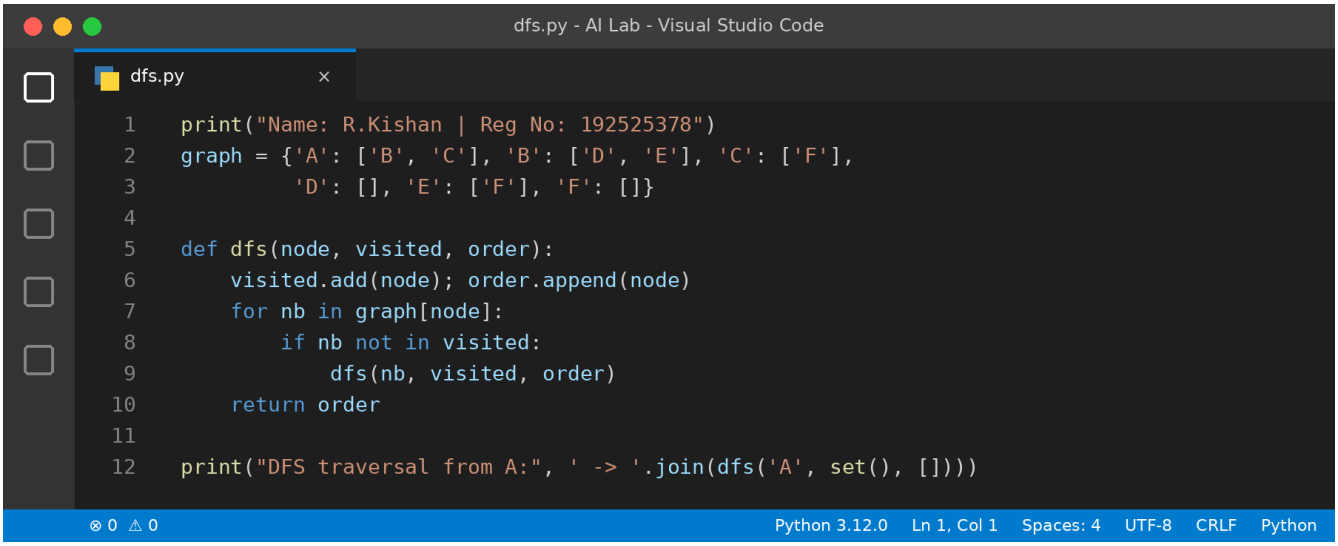
Aim

To write a Python program to traverse a graph using Depth First Search.

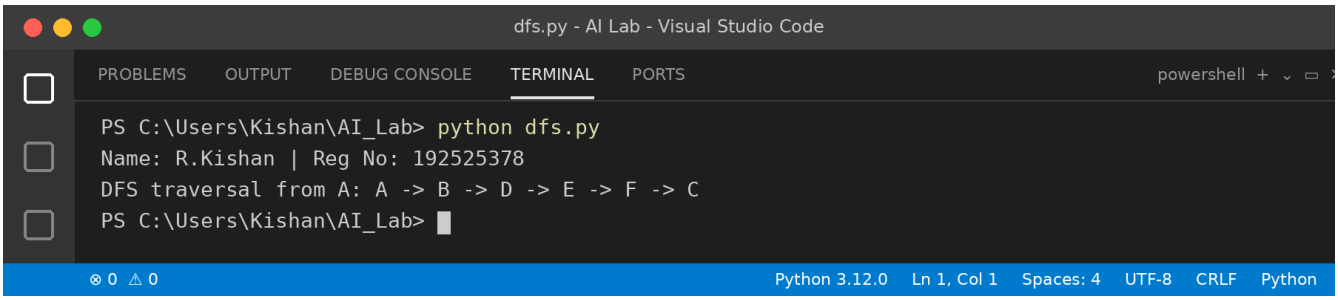
Procedure

1. Represent the graph as an adjacency list (dictionary).
2. Start from the source node, mark it visited and add it to the traversal order.
3. Recursively visit each unvisited neighbour of the current node.
4. Backtrack when a node has no unvisited neighbours.
5. Print the order in which the nodes were visited.

Program



Output



Result: Thus the Python program for Depth First Search (DFS) was executed successfully and the output was verified.

Program 9: Travelling Salesman Problem

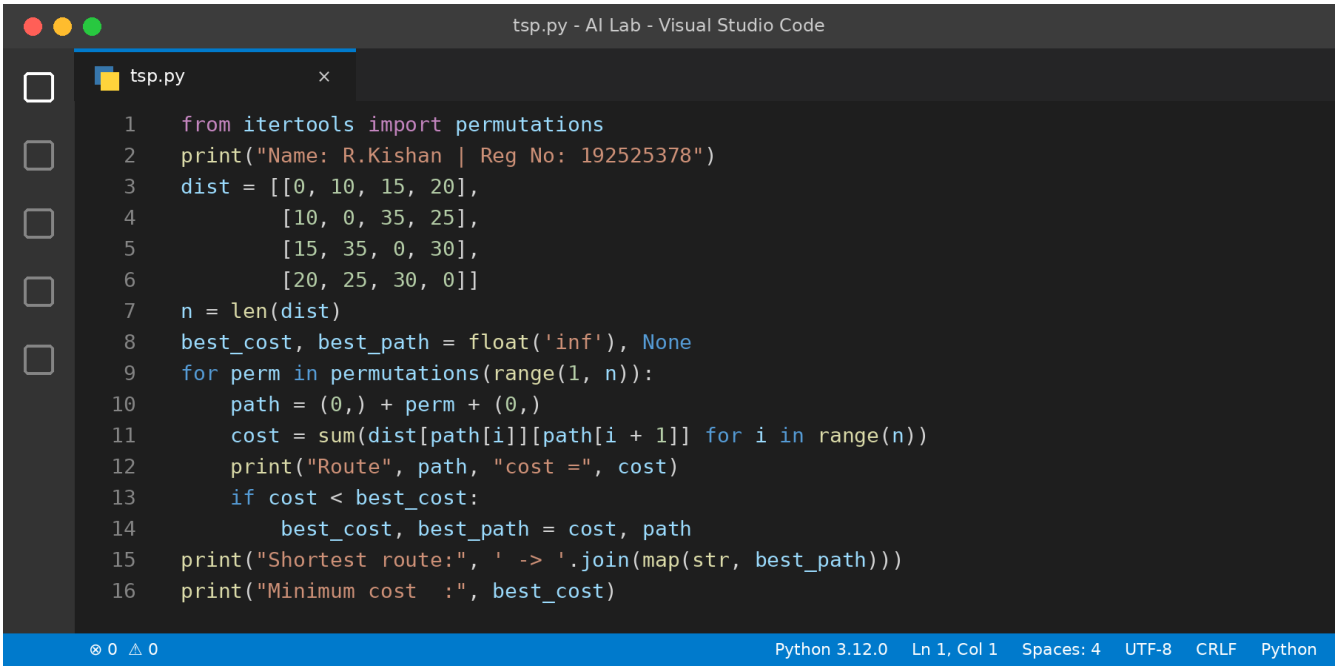
Aim

To write a Python program to find the shortest route that visits every city exactly once and returns to the starting city.

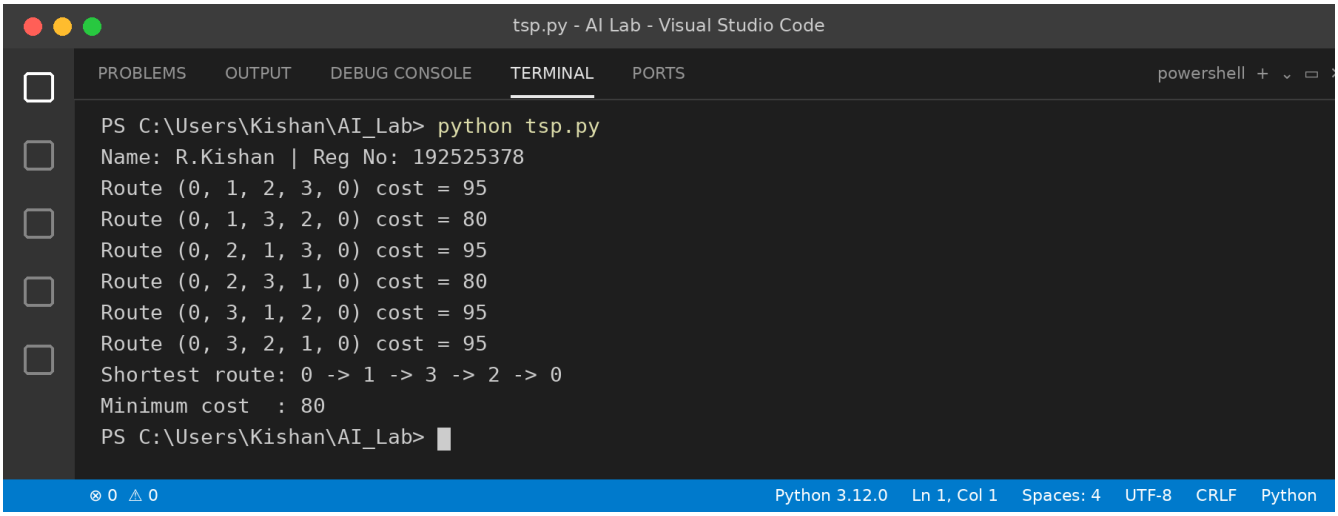
Procedure

1. Store the distances between the 4 cities in a distance matrix.
2. Fix city 0 as the start and generate all permutations of the remaining cities.
3. For each permutation compute the total distance of the round trip.
4. Keep track of the route having the minimum cost.
5. Print all routes with their costs, the shortest route and the minimum cost.

Program



Output



Result: Thus the Python program for Travelling Salesman Problem was executed successfully and the output was verified.

Program 10: A* Algorithm

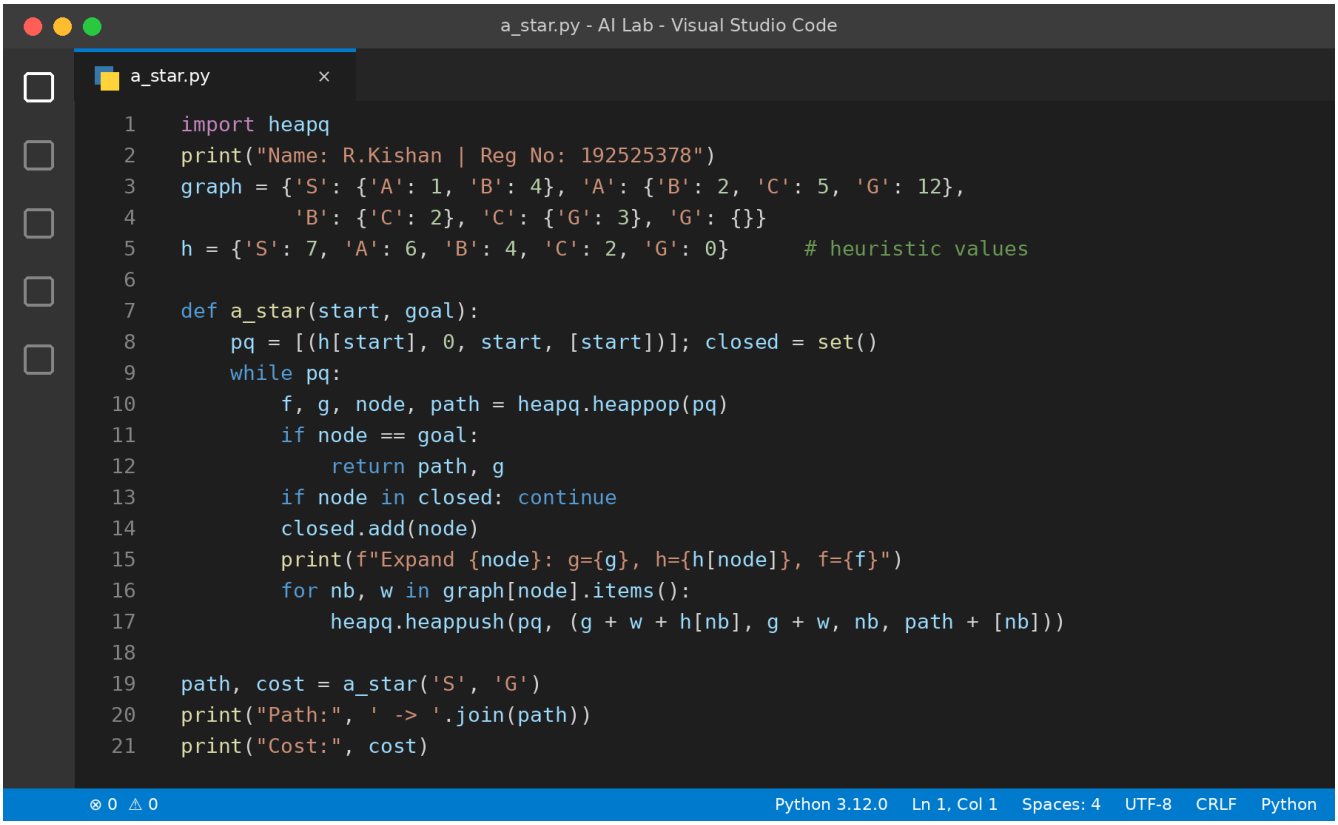
Aim

To write a Python program to find the least-cost path from a start node to a goal node using the A* algorithm.

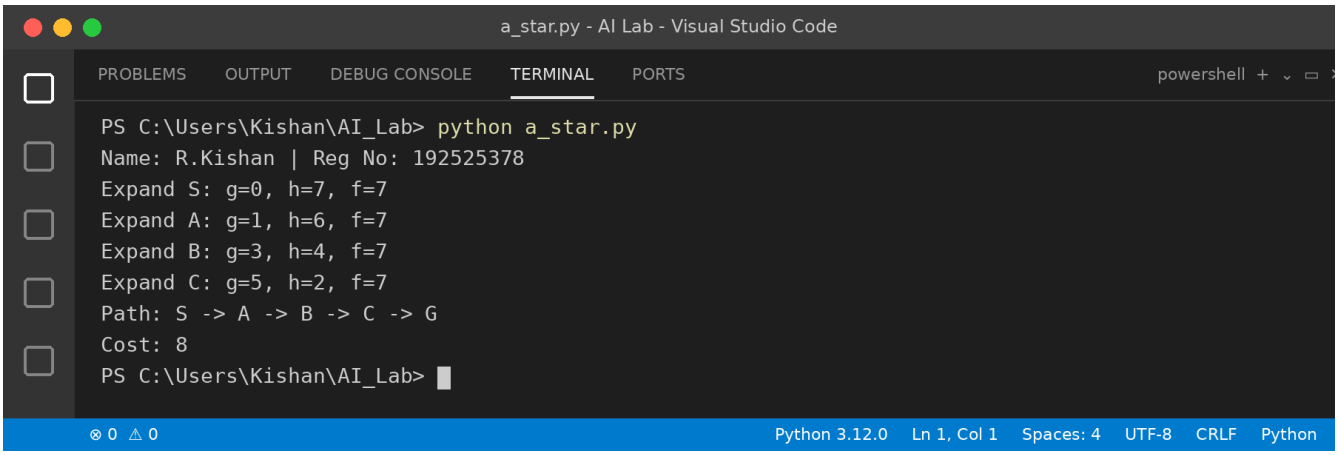
Procedure

1. Represent the weighted graph as a dictionary and provide a heuristic value h(n) for each node.
2. Calculate f(n) = g(n) + h(n), where g(n) is the cost from the start to n.
3. Keep the open nodes in a priority queue and expand the node with the smallest f(n).
4. Add the neighbours of the expanded node with their updated costs.
5. Stop when the goal is expanded; print the path and its total cost.

Program



Output



Result: Thus the Python program for A* Algorithm was executed successfully and the output was verified.

Program 11: Map Coloring (Constraint Satisfaction Problem)

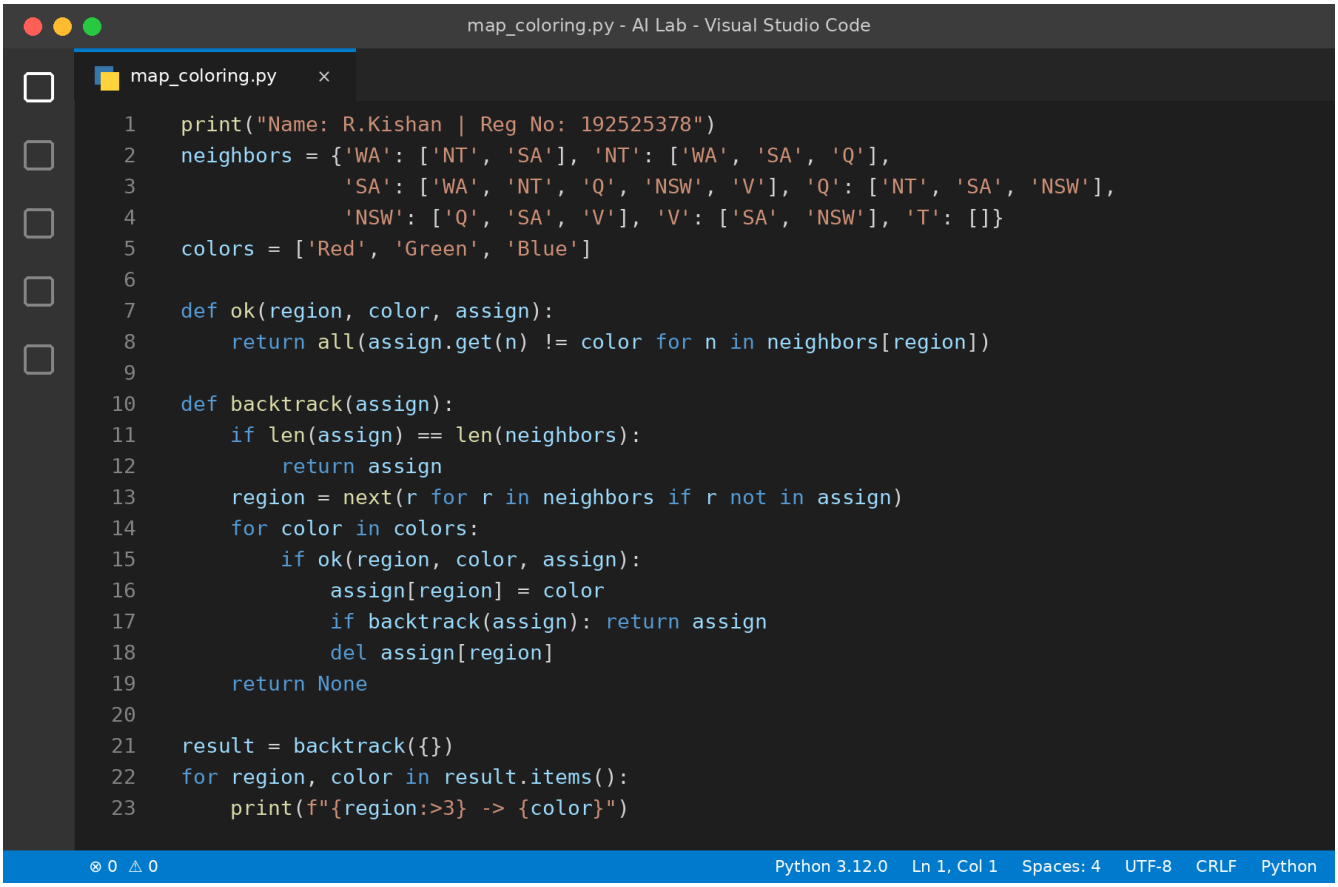
Aim

To write a Python program to colour the regions of the map of Australia with 3 colours so that no two adjacent regions have the same colour (CSP).

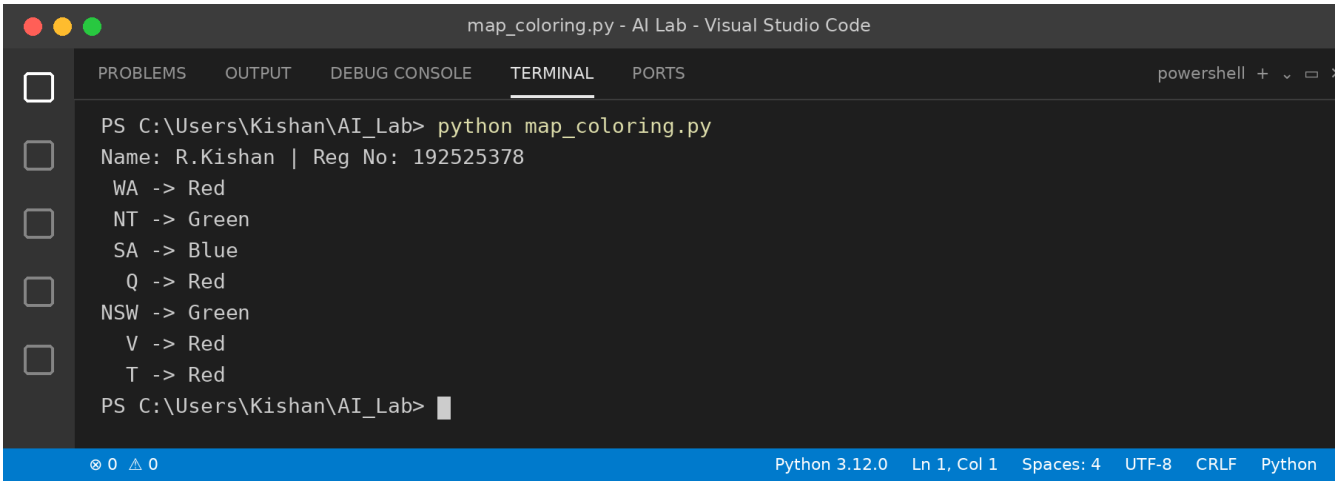
Procedure

1. Variables: the regions WA, NT, SA, Q, NSW, V, T. Domain: {Red, Green, Blue}.
2. Constraint: neighbouring regions must have different colours.
3. Pick an unassigned region and try each colour that satisfies the constraint with its neighbours.
4. If a region cannot be coloured, backtrack and change the previous assignment.
5. Print the final colour of every region.

Program



Output



Result: Thus the Python program for Map Coloring (Constraint Satisfaction Problem) was executed successfully and the output was verified.

Program 12: Tic Tac Toe Game

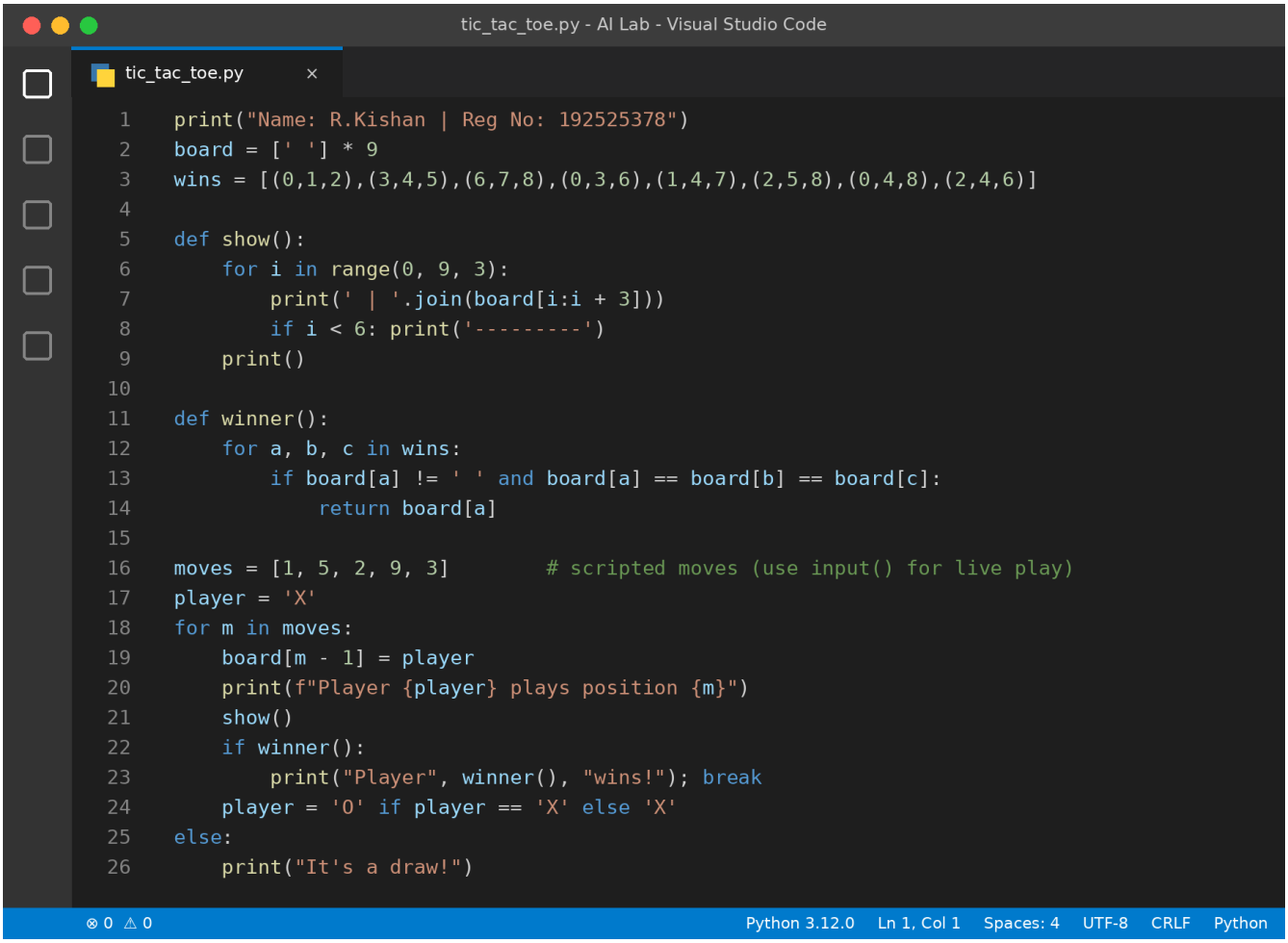
Aim

To write a Python program to play the Tic Tac Toe game between two players X and O.

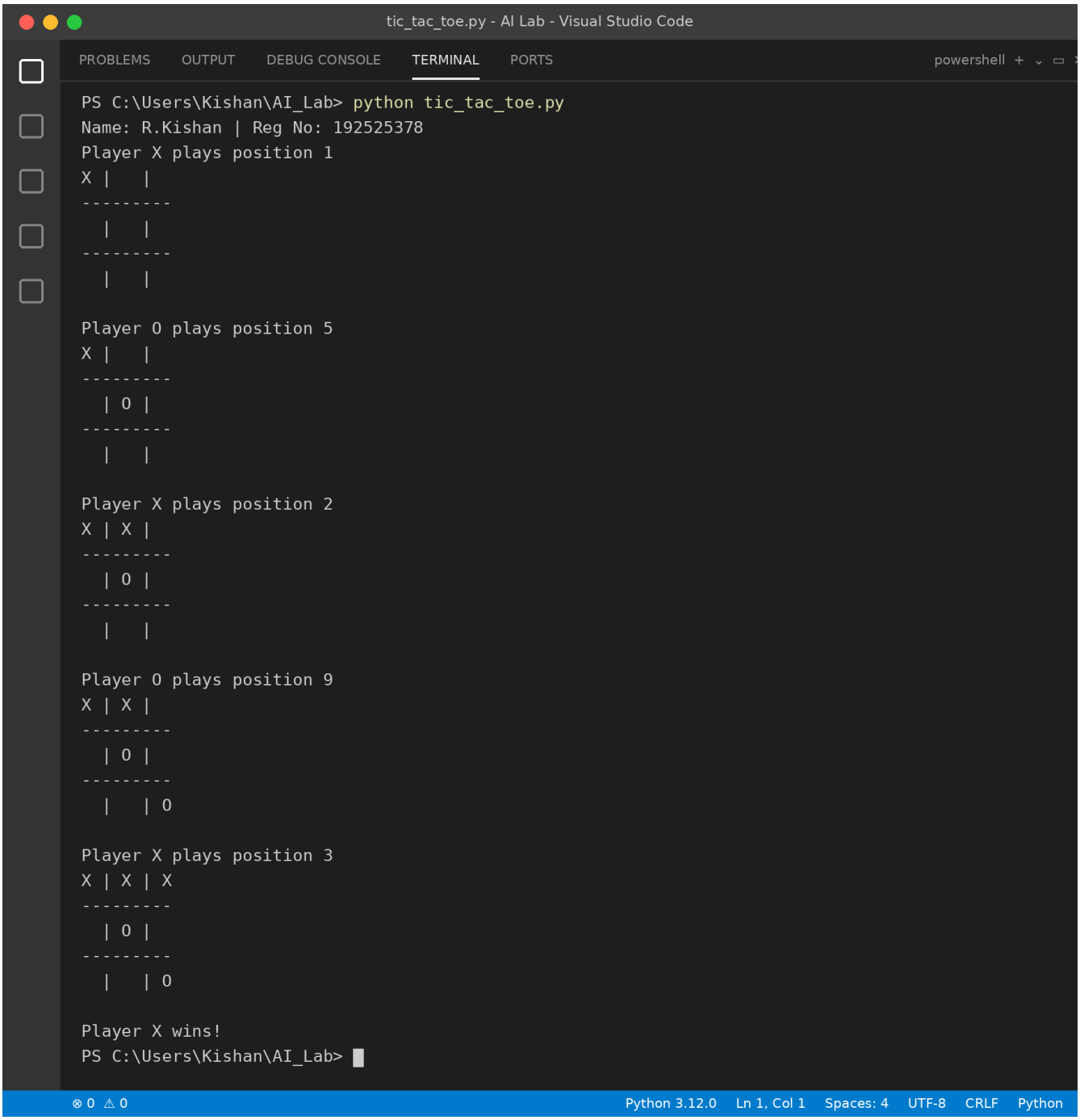
Procedure

1. Create a 3x3 board using a list of 9 positions and list the 8 winning lines.
2. Players X and O play alternately by choosing a position from 1 to 9.
3. After every move display the board and check whether a winning line is complete.
4. If a player completes a line, declare the winner; if the board is full, declare a draw.
5. (The moves are scripted here so that the output is reproducible.)

Program



Output



Result: Thus the Python program for Tic Tac Toe Game was executed successfully and the output was verified.

Program 13: Minimax Algorithm

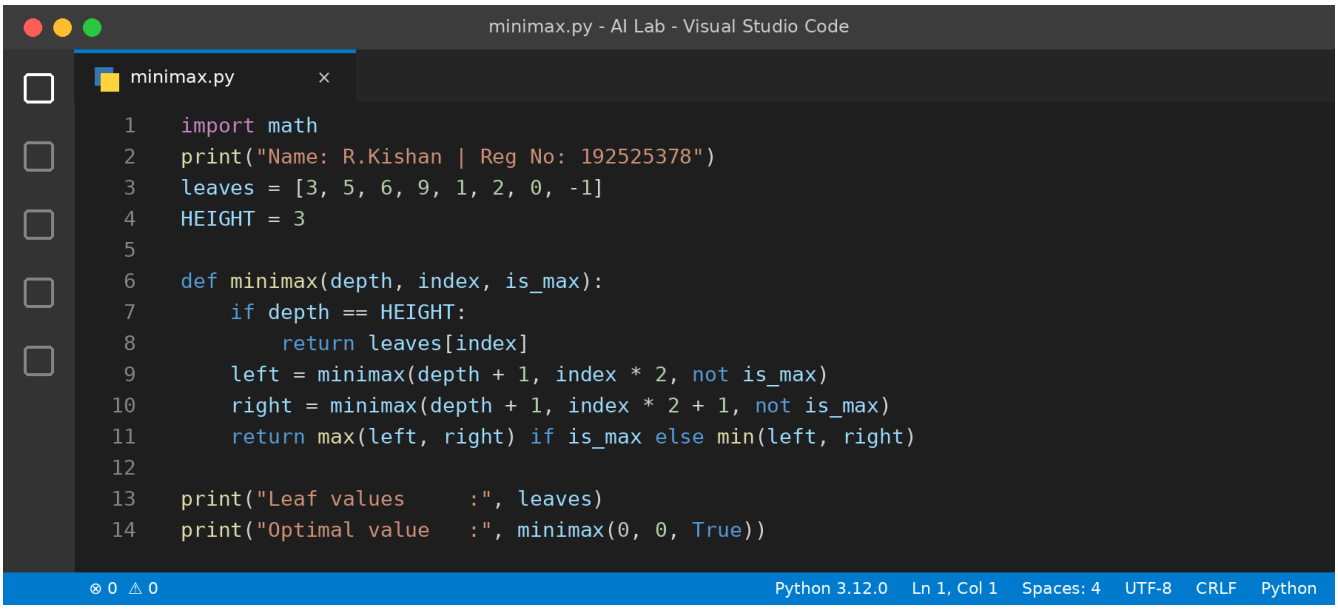
Aim

To write a Python program to implement the Minimax algorithm for a two-player game and find the optimal value of the game tree.

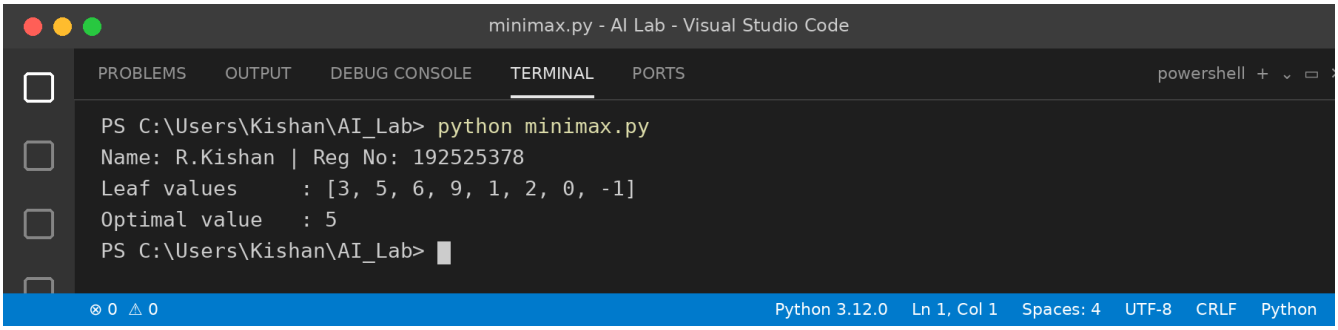
Procedure

1. Represent the game tree of height 3 using the list of leaf (utility) values.
2. At a MAX level choose the larger value of the two children; at a MIN level choose the smaller value.
3. Recursively evaluate the tree from the root down to the leaves and back up.
4. The value returned at the root is the optimal value for the MAX player.
5. Print the leaf values and the optimal value.

Program



Output



Result: Thus the Python program for Minimax Algorithm was executed successfully and the output was verified.

Program 14: Alpha-Beta Pruning

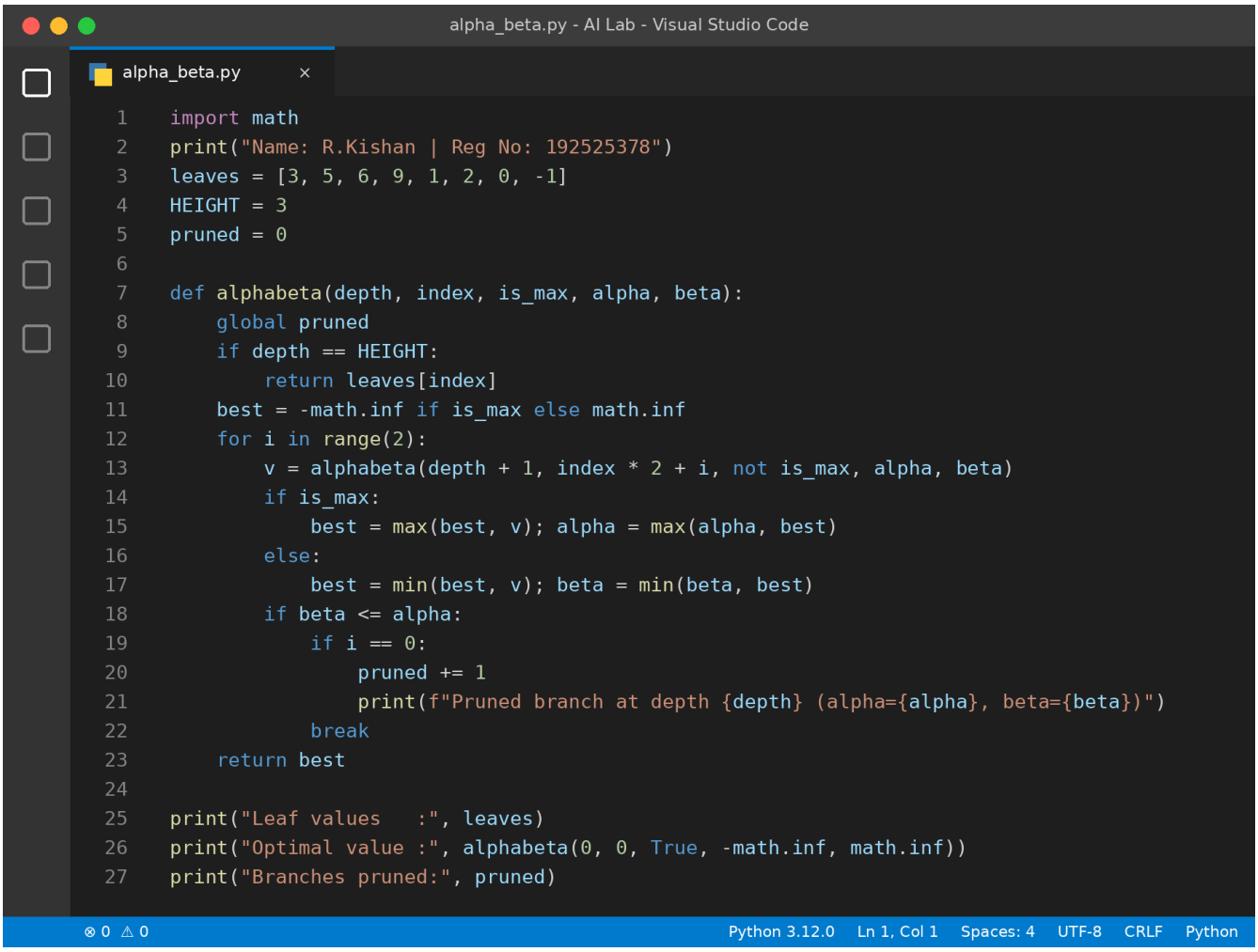
Aim

To write a Python program to implement Alpha-Beta pruning, an optimisation of Minimax that skips branches which cannot affect the result.

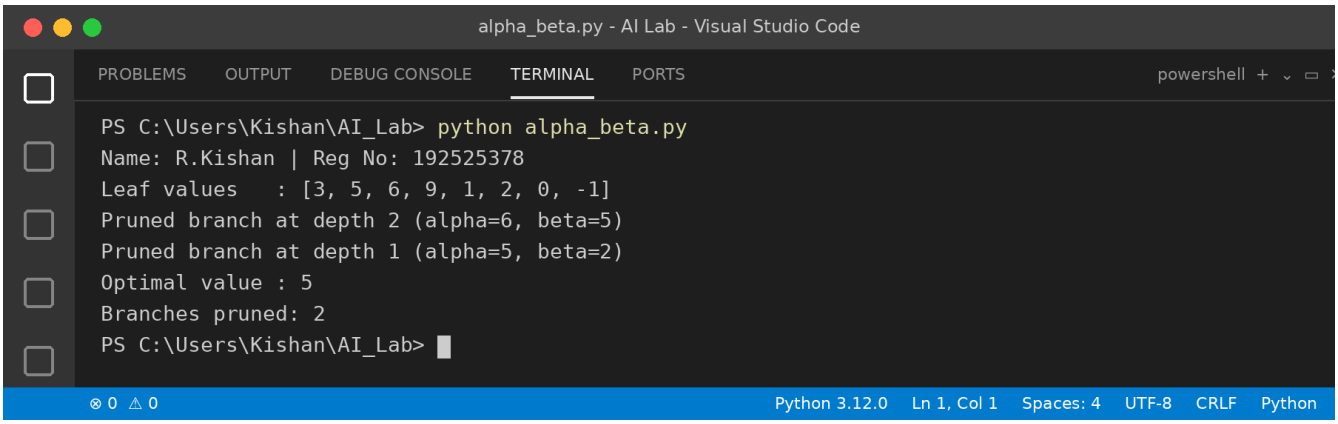
Procedure

1. Use the same game tree as Minimax and start with alpha = -infinity and beta = +infinity.
2. MAX nodes update alpha and MIN nodes update beta as children are evaluated.
3. Whenever beta <= alpha the remaining children of that node are pruned (skipped).
4. Return the value of the root node, which equals the Minimax value.
5. Print the pruned branches, the optimal value and the number of prunings.

Program



Output



Result: Thus the Python program for Alpha-Beta Pruning was executed successfully and the output was verified.

Program 15: Decision Tree (ID3)

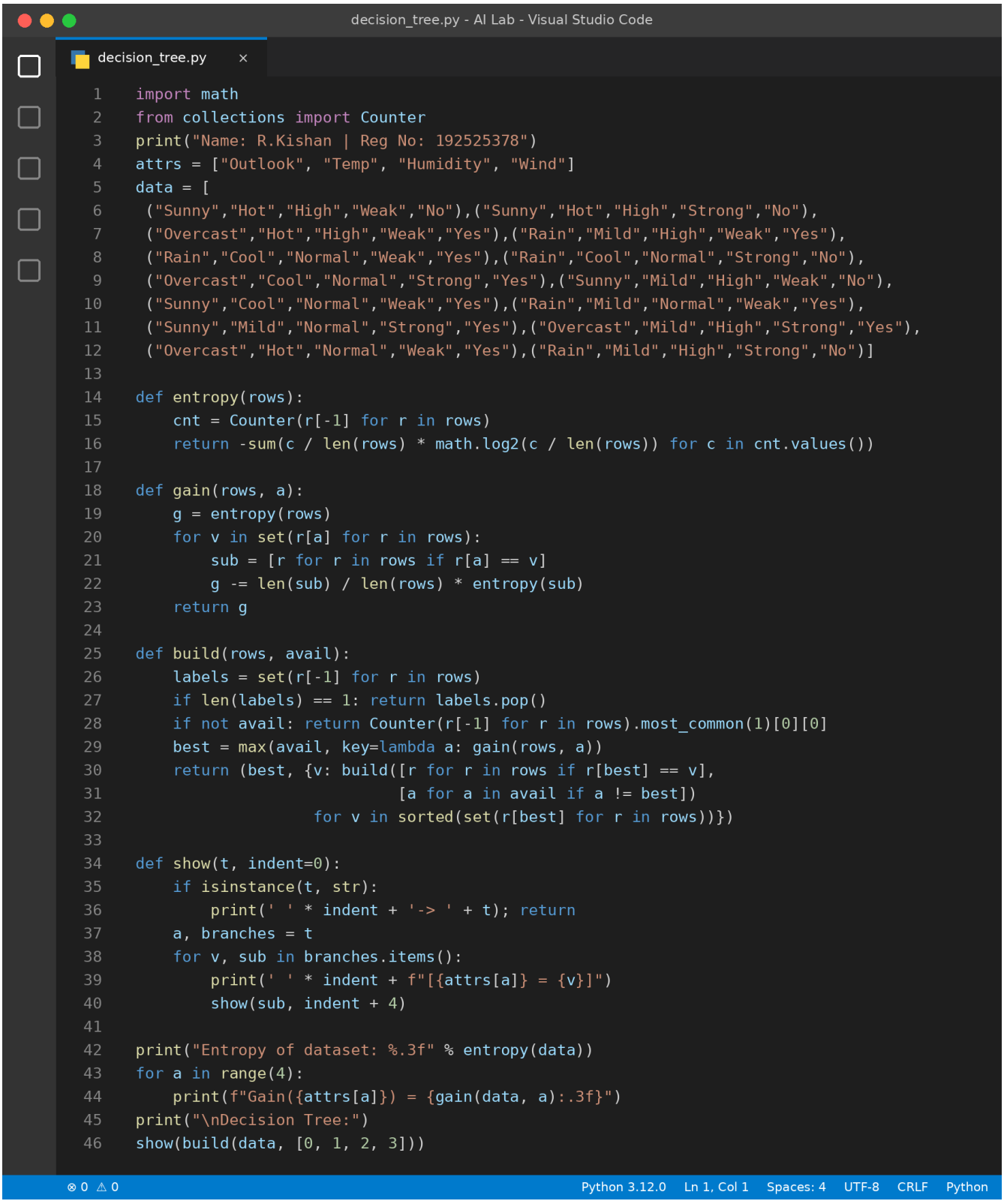
Aim

To write a Python program to build a Decision Tree using the ID3 algorithm (entropy and information gain) for the Play-Tennis dataset.

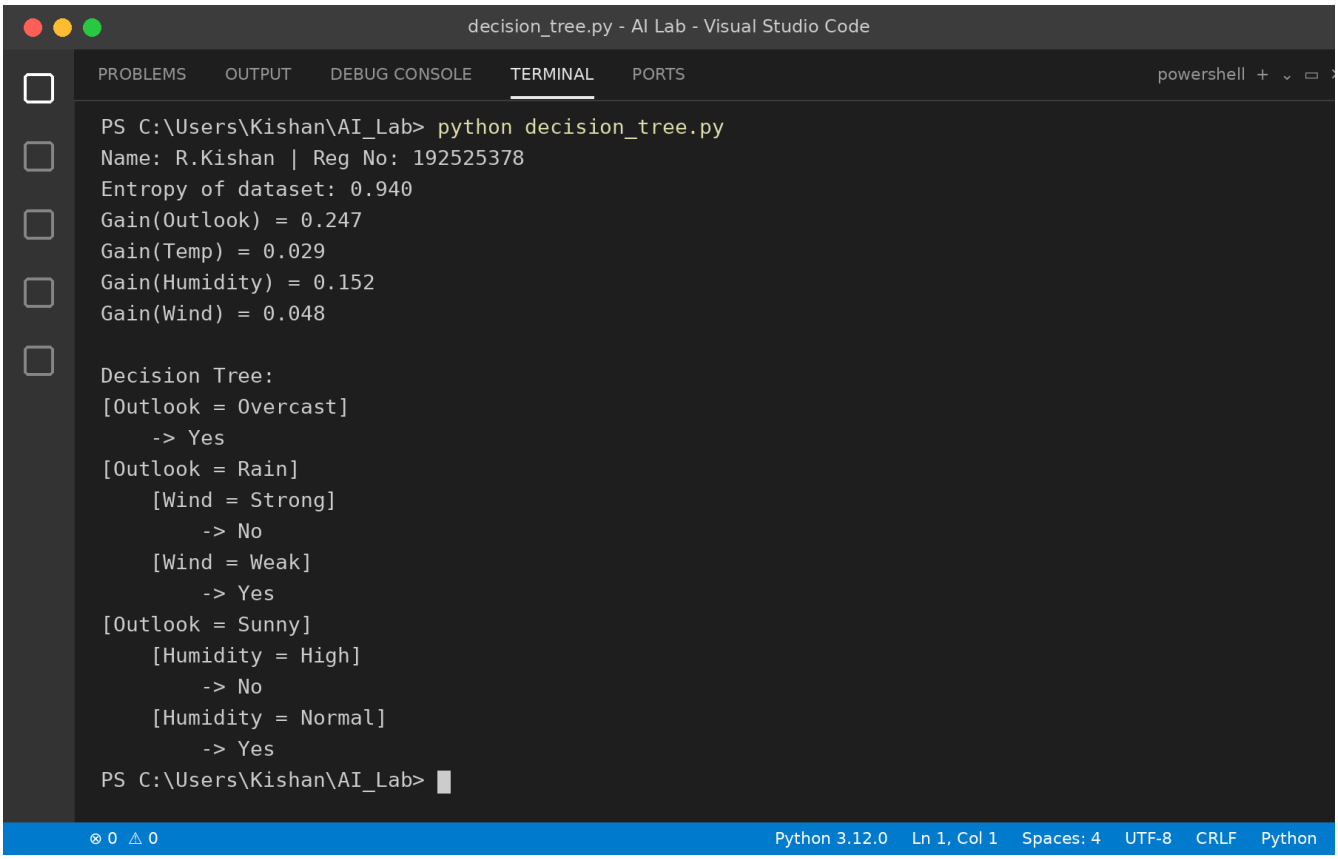
Procedure

1. Load the training data with attributes Outlook, Temp, Humidity and Wind and the class label PlayTennis.
2. Compute the entropy of the dataset and the information gain of every attribute.
3. Select the attribute with the highest gain as the root and split the data on its values.
4. Repeat recursively for every subset until all examples have the same class.
5. Print the resulting decision tree.

Program


decision_tree.py - AI Lab - Visual Studio Code
decision_tree.py
import math
from collections import Counter
print("Name: R.Kishan | Reg No: 192525378")
attrs = ["Outlook", "Temp", "Humidity", "Wind"]
data = [
("Sunny","Hot","High","Weak","No"),("Sunny","Hot","High","Strong","No"),
("Overcast","Hot","High","Weak","Yes"),("Rain","Mild","High","Weak","Yes"),
("Rain","Cool","Normal","Weak","Yes"),("Rain","Cool","Normal","Strong","No"),
("Overcast","Cool","Normal","Strong","Yes"),("Sunny","Mild","High","Weak","No"),
("Sunny","Cool","Normal","Weak","Yes"),("Rain","Mild","Normal","Weak","Yes"),
("Sunny","Mild","Normal","Strong","Yes"),("Overcast","Mild","High","Strong","Yes"),
("Overcast","Hot","Normal","Weak","Yes"),("Rain","Mild","High","Strong","No")]
def entropy(rows):
cnt = Counter(r[-1] for r in rows)
return -sum(c / len(rows) * math.log2(c / len(rows)) for c in cnt.values())
def gain(rows, a):
g = entropy(rows)
for v in set(r[a] for r in rows):
sub = [r for r in rows if r[a] == v]
g -= len(sub) / len(rows) * entropy(sub)
return g
def build(rows, avail):
labels = set(r[-1] for r in rows)
if len(labels) == 1: return labels.pop()
if not avail: return Counter(r[-1] for r in rows).most_common(1)[0][0]
best = max(avail, key=lambda a: gain(rows, a))
return (best, {v: build([r for r in rows if r[best] == v],
[a for a in avail if a != best])
for v in sorted(set(r[best] for r in rows))})
def show(t, indent=0):
if isinstance(t, str):
print(' ' * indent + '-> ' + t); return
a, branches = t
for v, sub in branches.items():
print(' ' * indent + f"[{attrs[a]} = {v}]")
show(sub, indent + 4)
print("Entropy of dataset: %.3f" % entropy(data))
for a in range(4):
print(f"Gain({attrs[a]}) = {gain(data, a):.3f}")
print("\nDecision Tree:")
show(build(data, [0, 1, 2, 3]))
⊗ 0 ⚠ 0
Python 3.12.0
Ln 1, Col 1
Spaces: 4
UTF-8
CRLF
Python

Output



Result: Thus the Python program for Decision Tree (ID3) was executed successfully and the output was verified.

Program 16: Feed-Forward Neural Network

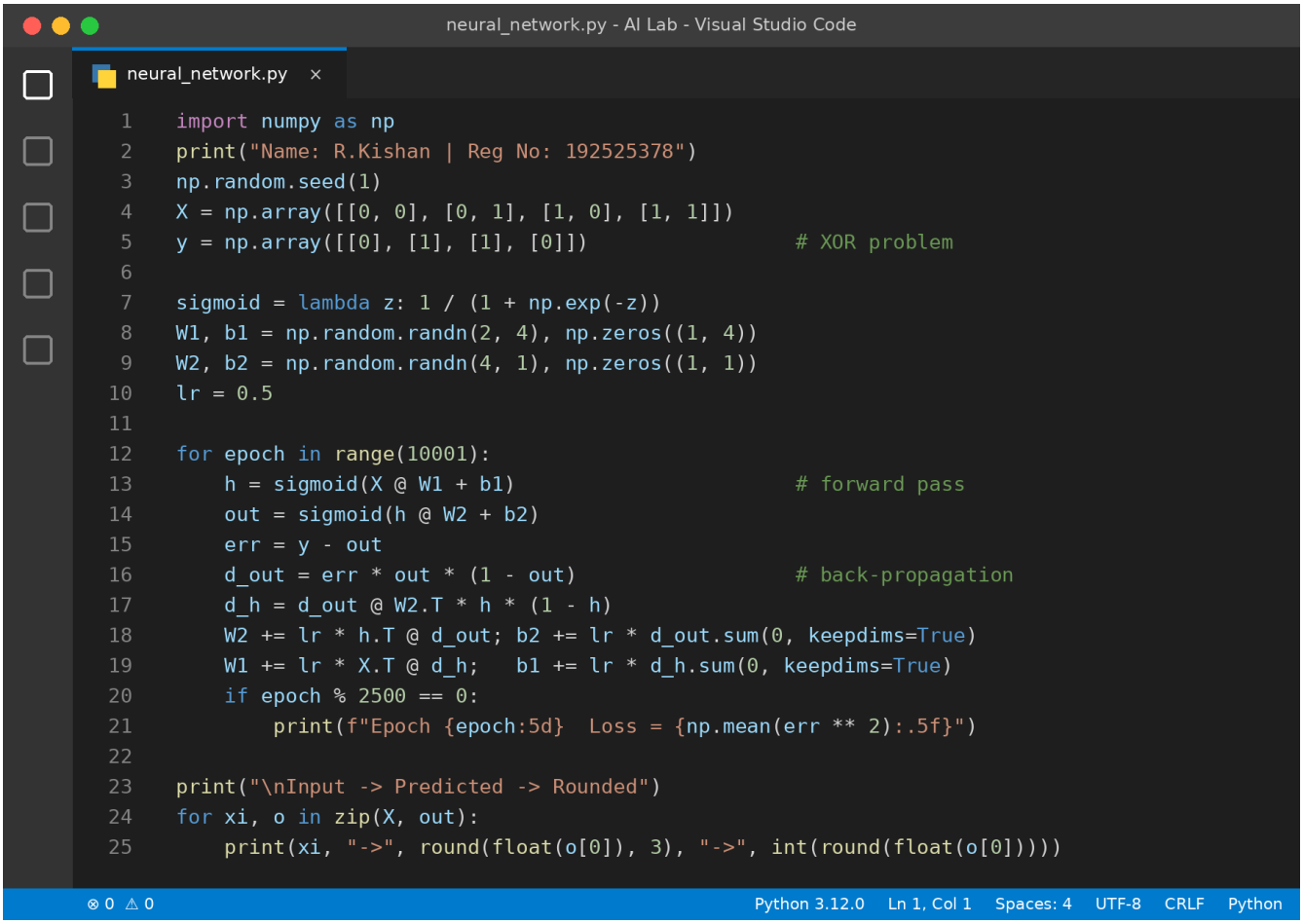
Aim

To write a Python program to implement a feed-forward neural network with back-propagation that learns the XOR function.

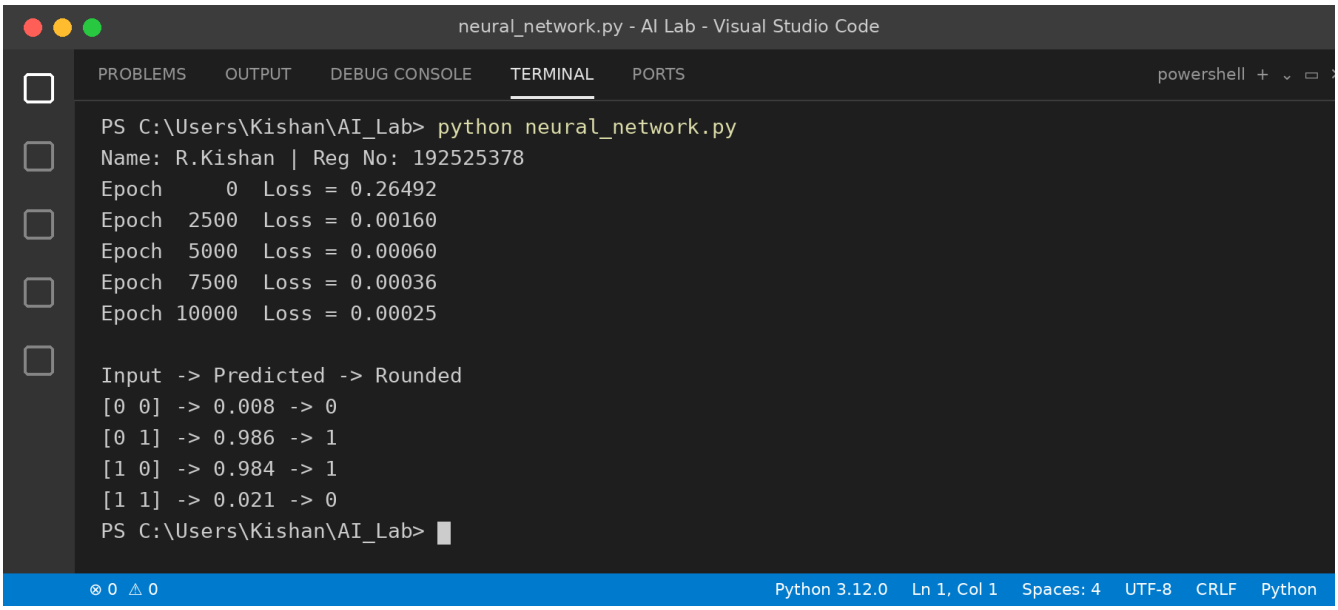
Procedure

1. Create the XOR inputs and target outputs; initialise weights randomly for a 2-4-1 network.
2. Forward pass: compute hidden-layer and output-layer activations with the sigmoid function.
3. Compute the error between the target and the output.
4. Back-propagate the error and update the weights and biases using gradient descent (learning rate 0.5).
5. Repeat for many epochs, print the loss, and finally print the predictions.

Program



Output



Result: Thus the Python program for Feed-Forward Neural Network was executed successfully and the output was verified.